

Fix submdspan for C++26

Document #: P3355R0
Date: 2024-07-14
Project: Programming Language C++
Library Evolution
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- [1 Revision History](#)
- [2 Abstract](#)
- [3 Motivation](#)
 - [3.1 Permit user-defined pair types](#)
 - [3.1.1 Requirement is currently an opt-in type list](#)
 - [3.1.2 Requirement should depend on structured binding instead](#)
 - [3.1.3 Yes, complex is a valid slice type; it already was](#)
 - [3.1.4 Slice elements are indices, so they have the same requirements](#)
 - [3.2 Preserve contiguity with compile-time extent](#)
 - [3.2.1 Vectorization example](#)
 - [3.2.2 Issues with C++ Working Draft](#)
 - [3.2.3 Why we don't try to fix `aligned\_accessor::offset`](#)
- [4 Wording](#)
- [5 Appendix A: A user-defined pair type](#)

§ 1 Revision History

§ 2 Abstract

We propose the following fixes to submdspan for C++26.

1. Permit user-defined pair types to be used as slices, rather than restricting the list to the opt-in set of types in **[tuple.like]**.

2. Change `layout_left`, `layout_right`, `layout_left_padded`, and `layout_right_padded` so that if a slice is `strided_slice` where `stride_type` models *integral-constant-like* and `stride_type::value` equals 1, then the same layout mapping type results as if the slice models *index-pair-like*<`index_type`>. This lets users express a slice with compile-time extent that does not cause common layouts to devolve into `layout_stride`.

§ 3 Motivation

§ 3.1 Permit user-defined pair types

§ 3.1.1 Requirement is currently an opt-in type list

Users sometimes want to express “a pair of integers” using types other than `std::tuple` or `std::pair`. For example, users might want their representation of a pair of two `integral_constant` to be an empty class, but a `std::tuple` of two elements is not empty. (This is because `get<0>` and `get<1>` must return references to their corresponding members.) However, the current requirements for a submdspan “pair of integers” slice, *index-pair-like*, include *pair-like*. This in turn includes *tuple-like*, an exposition-only concept which is an opt-in list of types: `array`, `complex`, `pair`, `tuple`, or `ranges::subrange` (see [\[tuple.like\]](#)). As a result, a user-defined type cannot model *tuple-like*.

§ 3.1.2 Requirement should depend on structured binding instead

The author of this proposal is a coauthor of P2630, and has asked the other authors of P2630 what they intended. The authors of P2630 always intended to support user-defined “pair-like” slice types. How they meant to define “pair-like” is that structured binding works for the slice type, and results in two elements, each of which is convertible to the layout mapping’s `index_type`. We can express “structured binding works” using the language of [\[decl.struct.bind\]](#) 4. The key is that `tuple_size_v<T>` is a “well-formed integral constant expression.” If it’s also equal to 2, then we have a “pair of indices” slice.

That last `convertible_to<index_type>` requirement doesn’t just support integers and *integral-constant-like* types; it also supports more interesting types like “strong typedefs” that wrap integers to give them meaning. A common use case for integer strong typedefs like this is to help prevent common bugs like mixing up loop indices. Here is a somewhat contrived example from the finite-element method for solving partial differential equations.

```
for (Element e = {}; e < num_elements; ++e) {
    for (FaceOffset fo = faces_begin[e]; fo < faces_end[e]; ++fo) {
        Face f = faces[fo]; // faces live in a flat array
        double sum = 0.0;
        for (QuadraturePointIndex q = {}; q <
num_quadrature_points[f]; ++q) {
            // Here, quadrature coefficients are the same for all faces.
            sum += quadrature_coefficient[q] *
```

```

value_at[quadrature_point_to_vertex(f, q)];
    }
    store_integration_result(f, sum);
}
}

```

§ 3.1.3 Yes, `complex` is a valid slice type; it already was

1. `complex` is a valid slice type; it was already before this proposal.
2. We don't propose to change that here, because it would break consistency between `mdspan`'s `operator[]` and the index type used by slices.

Some users might find it surprising that `complex<float>(1.25f, 3.75f)` is a valid slice with the same meaning as `tuple<int, int>{1, 4}`. This is true in the current Working Draft, because adoption of [P2819R2](#) (“Add tuple protocol to `complex`”) added `complex` to the opt-in list of *tuple-like* types. A valid “pair-like” slice has two requirements:

1. *pair-like* (that is, *tuple-like* with two elements); and
2. the elements are (implicitly) convertible to the `mdspan`'s `index_type`.

The type `complex<float>` is *pair-like* due to [P2819R2](#)'s adoption, and its element type `float` is implicitly convertible to `int`. Even though this proposal would change the *pair-like* requirement on slices, it would only strictly enlarge the set of valid slice types.

§ 3.1.4 Slice elements are indices, so they have the same requirements

The `convertible_to<index_type>` requirement exists for good reason: it's the same as the requirement on types that `mdspan`'s `operator[]` accepts. A slice represents a finite set of indices, and a “pair-like” slice specifically represents an interval with a finite lower and upper bound, both of which are indices. `x[1.25f]` works if `x` is an `mdspan`, an array, a vector, or even a raw array. Thus, we see no reason why `tuple{1.25f, 3.75f}` or even `complex<float>{1.25f, 3.75f}` shouldn't work as a slice.

One could carve out an exception like “but not `complex<T>` for any `T`.” However, that would have two issues. First, it would still permit types like `tuple<float>`. Second, it would permit user-defined `complex` types with floating-point element type, as long as those types define the tuple protocol. Support for user-defined `complex` types is a feature of the linear algebra proposal [P1673](#), which was adopted into the Working Draft. In general, while we might be able to exclude `complex` specifically, we don't have a way to define the open-ended set of types that “maybe shouldn't be treated as pair-like slices even though this works just fine syntactically.” For this reason, we do not propose treating `complex` differently than any other pair-like type.

If we wanted to carve out an exception like “convertible to `index_type`, but not a floating-point type,” then that would still permit class types that wrap a floating-point number, but are convertible to an integral type, like this.

```

struct WrappedFloat {
    float value_;

    operator int() const {
        return value_;
    }
};

```

Again, we find impossible to define the open-ended set of types that “maybe shouldn’t be used as array indices.”

The key rule is that the set of types permitted in slices should be the same as the set of types permitted in `mdspan::operator[]`. We should never change those two requirements separately. This rule would still let us carve out exceptions for the “pair-like” types themselves (e.g., we could forbid `complex`, even though we do not here), but not for the members of the pair-like types.

§ 3.2 Preserve contiguity with compile-time extent

§ 3.2.1 Vectorization example

Suppose that one wants to vectorize a 1-D array copy operation using `mdspan` and `aligned_accessor` (P2897). One has a `copy_8_floats` function that optimizes the special case of copying a contiguous array of 8 floats, where the start of the array is aligned to $8 * \text{sizeof}(\text{float})$ (32) bytes. In practice, plenty of libraries exist to optimize 1-D array copy. This is just an example that simplifies the use cases for explicit 8-wide SIMD enough to show in a brief proposal.

```

template<class ElementType, size_t ext, size_t byte_alignment>
using aligned_array_view = mdspan<ElementType,
    extents<int, ext>, layout_right,
    aligned_accessor<ElementType, byte_alignment>>;

void
copy_8_floats(aligned_array_view<const float, 8, 32> src,
    aligned_array_view<float, 8, 32> dst)
{
    // One would instead use a hardware instruction for aligned
copy,
    // or a "simd" or "unroll" pragma.
    for (int k = 0; k < 8; ++k) {
        dst[k] = src[k];
    }
}

```

The natural next step would be to use `copy_8_floats` to implement copying 1-D float arrays by the usual “strip-mining” approach.

```

template<class ElementType>
using array_view = mdspan<ElementType, dims<1, int>>;

void slow_copy(array_view<const float> src, array_view<float> dst)

```

```

    {
        assert(src.extent(0) == dst.extent(0));
        for (int k = 0; k < src.extent(0); ++k) {
            dst[k] = src[k];
        }
    }

template<size_t vector_length>
void strip_mined_copy(
    aligned_array_view<const float, dynamic_extent,
        vector_length * sizeof(float)> src,
    aligned_array_view<float, dynamic_extent,
        vector_length * sizeof(float)> dst)
{
    assert(src.extent(0) == dst.extent(0));
    assert(src.extent(0) % vector_length == 0);

    for (int beg = 0; beg < src.extent(0); beg += vector_length) {
        constexpr auto one = std::integral_constant<int, 1>{};
        constexpr auto vec_len = std::integral_constant<int,
vector_length>{};

        // Using strided_slice lets the extent be a compile-time
constant.
        // tuple{beg, beg + vector_length} would result in
dynamic_extent.
        constexpr auto vector_slice =
            strided_slice{.offset=dst_lower, .extent=vector_length,
.stride=one};

        // PROBLEM: Current wording makes this always layout_stride,
// but we know that it could be layout_right.
        auto src_slice = submdspan(src, vector_slice);
        auto dst_slice = submdspan(dst, vector_slice);

        copy_8_floats(src_slice, dst_slice);
    }
}

void copy(array_view<const float> src, array_view<float> dst)
{
    assert(src.extent(0) == dst.extent(0));
    constexpr int vector_length = 8;

    // Handle possibly unaligned prefix of less than vector_length
elements.
    auto aligned_starting_index = [](auto* ptr) {
        constexpr auto v = static_cast<unsigned>(vector_length);
        auto ptr_value = reinterpret_cast<uintptr_t>(ptr_value);
        auto remainder = ptr_value % v;
        return static_cast<int>(ptr_value + (v - remainder) % v);
    };
    int src_beg = aligned_starting_index(src.data());
    int dst_beg = aligned_starting_index(dst.data());
    if (src_beg != dst_beg) {

```

```

        slow_copy(src, dst);
        return;
    }
    slow_copy(submdspan(src, tuple{0, src_beg}),
              submdspan(dst, tuple{0, dst_beg}));

    // Strip-mine the aligned vector_length segments of the array.
    int src_end = (src.size() / vector_length) * vector_length;
    int dst_end = (dst.size() / vector_length) * vector_length;
    strip_mined_copy<8>(submdspan(src, tuple{src_beg, src_end}),
                       submdspan(dst, tuple{dst_beg, dst_end}));

    // Handle suffix of less than vector_length elements.
    slow_copy(submdspan(src, tuple{src_end, src.extent(0)}),
              submdspan(dst, tuple{dst_end, dst.extent(0)}));
}

```

The `strip_mined_copy` function must use `strided_slice` to get slices of 8 elements at a time, rather than `tuple`. This ensures that the resulting extent is a compile-time constant 8, even though the slice starts at a run-time index `beg`.

§ 3.2.2 Issues with C++ Working Draft

The current C++ Working Draft has two issues that hinder optimization of the above code.

1. The above `submdspan` results always have `layout_stride`, even though we know that they are contiguous and thus should have `layout_right`.
2. The `submdspan` operations in `strip_mined_copy` should result in `aligned_accessor` with 32-byte alignment, but instead give `default_accessor`. This is because `aligned_accessor`'s `offset` member function takes the offset as a `size_t`. This discards compile-time information, like knowing that the offset is divisible by some overalignment factor.

This proposal fixes (1) for `layout_left`, `layout_right`, `layout_left_padded`, and `layout_right_padded`. We can do that without breaking changes, because C++26 is still a working draft. This proposal does *not* fix (2), because that would require a breaking change to both the layout mapping requirements and the accessor requirements, and because it would complicate both of them quite a bit.

§ 3.2.3 Why we don't try to fix `aligned_accessor::offset`

Regarding (2), [\[mdspan.submdspan.submdspan\] 6](#) says that `submdspan(src, slices...)` has effects equivalent to the following.

```

auto sub_map_offset = submdspan_mapping(src.mapping(), slices...);
return mdspan(src.accessor().offset(src.data(),
sub_map_offset.offset),
              sub_map_offset.mapping,
              AccessorPolicy::offset_policy(src.accessor()));

```

The problem is `AccessorPolicy::offset_policy(src.accessor())`. The type `offset_policy` is the wrong type in this case, `default_accessor<const float>` instead of `aligned_accessor<const float, 32>`. If we want an offset with suitable compile-time alignment to have a different accessor type, then we would need at least the following changes.

1. The Standard Library would need a new type that represents the product of a compile-time integer (that is, an *integral-constant-like* type) and a “run-time” integer (an *integral-not-bool* type). It would need overloaded arithmetic operators that preserve this product form as much as possible. For example, $8x + 4$ for a run-time integer x should result in $4y$ where $y = 2x + 1$ is a run-time integer.
2. At least the Standard layout mappings’ `operator()` would need to compute with these types and return them if possible. The layout mapping requirements would thus need to change, as currently `operator()` must return `index_type` (see [\[\[mdspan.layout.reqmts\]\] 7](#)).
3. `aligned_accessor::offset` would need an overload taking a type that expresses the product of a compile-time integer (of suitable alignment) and a run-time integer. The accessor requirements [\[\[mdspan.accessor.reqmts\]\]](#) may also need to change to permit this.
4. The definition of `submdspan` would need some way to get the accessor type corresponding to the new offset overload, instead of `aligned_accessor::offset_policy` (which in this case is `default_accessor`).

The work-around is to convert the result of `submdspan` by hand to use the desired accessor. In the above copy example, one would replace the line

```
copy_8_floats(src_slice, dst_slice);
```

with the following, that depends on `aligned_accessor`’s explicit constructor from `default_accessor`.

```
copy_8_floats(aligned_array_view<const float, 8, 32>{src},
              aligned_array_view<float, 8, 32>{dst});
```

Given that this work-around is easy to do, should only be needed for a few special cases, and avoids a big design change to the accessor policy requirements, we don’t propose trying to fix this issue in the C++ Working Draft.

§ 4 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

In [\[mdspan.syn\]](#) (“Header `<mdspan>` synopsis”), replace the *index-pair-like* definition with the following. (The only line changed is *pair-like*`<T>`

&&.)

```
template<class T, class IndexType>
concept index-pair-like = // exposition only
    is_integral_v<decltype(tuple_size_v<T>)> &&
    (tuple_size_v<T> == 2) &&
    convertible_to<tuple_element_t<0, T>, IndexType> &&
    convertible_to<tuple_element_t<1, T>, IndexType>;
```

Throughout [mdspan.sub], wherever the text says

“ S_k models *index-pair-like*<index_type> or *is_convertible_v*< S_k , full_extent_t> is true,”

replace it with

“ S_k is a specialization of *strided_slice* where *stride_type* models *integral-constant-like* and *stride_type::value* equals 1, S_k models *index-pair-like*<index_type>, or *is_convertible_v*< S_k , full_extent_t> is true.”

Apply the analogous transformation if the text says S_p or S_0 , but is otherwise the same. Make this set of changes in the following places.

- [mdspan.sub.map.left] (1.3.2), (1.4), (1.4.1), and (1.4.3);
- [mdspan.sub.map.right] (1.3.2), (1.4), (1.4.1), and (1.4.3);
- [mdspan.sub.map.leftpad] (1.3.2), (1.4), (1.4.1), and (1.4.3); and
- [mdspan.sub.map.rightpad] (1.3.2), (1.4), (1.4.1), and (1.4.3).

§ 5 Appendix A: A user-defined pair type

This section is not normative. It shows an example of a user-defined pair type `user_pair` where elements of the pair that are empty are not stored. This proposal would permit `user_pair<First, Second>` to be used as a slice type with the same meaning as `std::tuple<First, Second>`. This [Compiler Explorer link](#) demonstrates the code below with four compilers: GCC 14.1, Clang 18.1.0, MSVC v19.40 (VS17.10), and nvc++ 24.5.

```
#include <cassert>
#include <tuple>
#include <type_traits>

using std::size_t;

namespace user {

// User-defined pair type that only stores non-empty elements.
template<class First, class Second,
```



```

    bool FirstIsEmpty = std::is_empty_v<First>,
    bool SecondIsEmpty = std::is_empty_v<Second>>
struct user_pair;

template<class First, class Second>
struct user_pair<First, Second, true, true> {};

template<class First, class Second>
struct user_pair<First, Second, false, true> {
    First first;
};

template<class First, class Second>
struct user_pair<First, Second, true, false> {
    Second second;
};

template<class First, class Second>
struct user_pair<First, Second, false, false> {
    First first;
    Second second;
};

} // namespace user

namespace std {

template<class First, class Second>
struct tuple_size<user::user_pair<First, Second>>
    : std::integral_constant<size_t, 2> {};

template<class First, class Second>
struct tuple_element<0, user::user_pair<First, Second>> {
    using type = First;
};

template<class First, class Second>
struct tuple_element<1, user::user_pair<First, Second>> {
    using type = Second;
};

} // namespace std

template<size_t Index, class First, class Second>
decltype(auto) get(const user::user_pair<First, Second>& t) {
    if constexpr (Index == 0) {
        if constexpr (std::is_empty_v<First>) {
            return First{};
        }
        else {
            return static_cast<const First&>(t.first);
        }
    }
    else if constexpr (Index == 1) {
        if constexpr (std::is_empty_v<Second>) {

```

```

        return Second{};
    }
    else {
        return static_cast<const Second&>(t.second);
    }
}
else {
    static_assert(Index < 2u);
}
}

template<size_t Index, class First, class Second>
decltype(auto) get(user::user_pair<First, Second>& t) {
    if constexpr (Index == 0) {
        if constexpr (std::is_empty_v<First>) {
            return First{};
        }
        else {
            return static_cast<First&>(t.first);
        }
    }
    else if constexpr (Index == 1) {
        if constexpr (std::is_empty_v<Second>) {
            return Second{};
        }
        else {
            return static_cast<Second&>(t.second);
        }
    }
    else {
        static_assert(Index < 2u);
    }
}

template<size_t Index, class First, class Second>
decltype(auto) get(user::user_pair<First, Second>&& t) {
    if constexpr (Index == 0) {
        if constexpr (std::is_empty_v<First>) {
            return First{};
        }
        else {
            return std::move(t.first);
        }
    }
    else if constexpr (Index == 1) {
        if constexpr (std::is_empty_v<Second>) {
            return Second{};
        }
        else {
            return std::move(t.second);
        }
    }
    else {
        static_assert(Index < 2u);
    }
}

```

```

    }

    template<size_t Value>
    struct Empty {};

    int main() {
        static_assert(std::is_empty_v<Empty<1>>);
        static_assert(std::is_empty_v<Empty<2>>);
        static_assert(std::is_empty_v<user::user_pair<Empty<1>,
Empty<2>>>);

        using type = user::user_pair<int, Empty<2>>;

        static_assert(std::is_same_v<decltype(std::tuple_size_v<type>),
const size_t>);
        static_assert(std::is_same_v<std::tuple_element_t<0, type>,
int>);
        static_assert(std::is_same_v<std::tuple_element_t<1, type>,
Empty<2>>);

        {
            type t{1};
            static_assert(std::is_same_v<decltype(get<0>(t)), int&>);
            static_assert(std::is_same_v<decltype(get<1>(t)), Empty<2>>);
            assert(get<0>(t) == 1);
            get<0>(t) = 2;
            assert(get<0>(t) == 2);
        }
        {
            const type t{1};
            static_assert(std::is_same_v<decltype(get<0>(t)), const
int&>);

            static_assert(std::is_same_v<decltype(get<1>(t)), Empty<2>>);
            assert(get<0>(t) == 1);
        }
        {
            static_assert(std::is_same_v<decltype(get<0>(type{1})),
int&&>);
            static_assert(std::is_same_v<decltype(get<1>(type{1})),
Empty<2>>);
            assert(get<0>(type{1}) == 1);
        }

        return 0;
    }

```